Archivos, árboles y hashing

Archivos

Capítulo 2, archivos, estructuras y operaciones básicas

Un archivo es una colección de registros semejantes guardados en dispositivos de almacenamiento secundario. Por lo tanto, un archivo es una estructura de datos homogénea que se caracteriza por el crecimiento y las modificaciones que se efectúan sobre ellos.

A la hora de trabajar los archivos, hay que considerar 2 aspectos, visión **física** y **lógica**.

- **Archivo físico:** reside en *memoria secundaria* y es administrado por el S.O
- **Archivo lógico:** es usado por el algoritmo. Cuando el algoritmo necesita operar con un archivo, genera una conexión entre el S.O, el cual se encargará de la administración, esto se conoce como *independencia física.*

Un archivo contiene elementos del mismo tipo, haciéndolo una **estructura homogénea.**

Acceso a los datos

- **Secuencial:** el acceso a cada elemento de datos se realiza luego de haber accedio a su inmediato anterior. Es de O(N) y la cantidad de lecturas en promedio es n/2.
- **Secuencial indizado:** el acceso a los elementos de un archivo se realiza teniendo presente algún tipo de organización previa.
- **Directo:** es posible recuperar un elemento de dato de un archivo con un solo acceso.

Tiempo de ejecución acceso a datos

Tipo de acceso	Orden de ejecución	Peor caso	Caso promedio
Secuencial	**O(N)** para encontrar un dato específico, se inicia desde el principio del archivo y **revisa cada elemento en orden** hasta que encuentres el deseado	Si el elemento que buscás está al final del archivo, o si el elemento no existe, habrá que leer **todos los N elementos**. Esto representa el **peor escenario**	Podría encontrarse en la mitad del archivo, por lo tanto **N/2**
Secuencial indizado	**O(log n)** (logarítmica). Los índices suelen implementarse como **árboles balanceados**, donde el número de operaciones necesarias para encontrar un elemento crece de manera logarítmica con el número total de elementos (n) en el índice. En cada paso de la búsqueda en el árbol, el número de elementos a considerar se reduce drásticamente.		
Directo	**O(1) en el mejor de los casos, O(N) en el peor de los casos**	**O(1) tiempo constante,** ocurre cuando la función de hashing mapea la clave directamente a una posición única y disponible, o cuando el registro se encuentra en la primera posición calculada. Es decir, se requiere una cantidad	**O(N) tiempo lineal,** este escenario se da principalmente debido a las **colisiones**. Se deberá alguna **técnicas de resolución de colisiones.** En el peor de los casos el acceso al registro podría requerir examinar una gran parte o incluso la totalidad (N) de los elementos.

		fija y mínima de operaciones (generalmente una sola lectura/escritura) para acceder al registro.	

Buffers

Área de memoria **RAM temporal**, gestionada por el **Sistema Operativo (SO)**, que sirve como intermediario para **optimizar la transferencia de datos** entre componentes que operan a distintas velocidades (ej., programas y memoria secundaria). Agrupa las operaciones de E/S para reducir accesos lentos.

Está en memoria principal o sea RAM.

Operaciones básicas

Las operaciones con archivos en un programa se realizan a través de un **nombre lógico**, que representa una variable de tipo archivo a la que se le ha asignado una conexión con un archivo físico. Estas operaciones no actúan directamente sobre la memoria secundaria, sino que leen y/o escriben sobre **buffers** relacionados con los archivos en la RAM.

Operación	Descripción
`rewrite(nombre_logico);`	Función de escritura la cual se usa para **crear un nuevo archivo** o para sobrescribir uno existente, posicionando el puntero al inicio.
`reset(nombre_logico);`	Función de **lectura y escritura** que se usa para **abrir un archivo existente**, posicionando el puntero al inicio para permitir la lectura o escritura desde el comienzo.
`close(nombre_logico);`	Indica que **no se trabajará más con el archivo**. Su ejecución implica, entre otras cosas, la liberación de los buffers asociados y la escritura de cualquier dato pendiente desde el buffer al disco. Puede también implicar poner una marca de **EOF (end of file)** al final del archivo si se estaba escribiendo en él.
`read(nombre_logico, variable);`	Se utiliza para **leer un elemento (registro) de datos** desde el archivo al que apunta `nombre_logico`. `variable` contendrá el siguiente elemento de datos del archivo, y el puntero de lectura/escritura del archivo avanzará al siguiente elemento.
`write(nombre_logico, variable);`	Esta operación se utiliza para **escribir el contenido de una** `variable` desde el programa hacia el archivo al que apunta `nombre_logico`. Después de una escritura, el elemento se añade al archivo, y el puntero de lectura/escritura del archivo avanza.
`eof(nombre_logico);`	Función usada para verificar si el puntero de lectura/escritura del archivo alcanzó el **EOF**. Devuelve un valor booleano indicándolo
`fileSize(nombre_logico);`	Función que devuelve el **tamaño total del archivo** al que se refiere `nombre_logico`
`filePos(nombre_logico);`	Fevuelve la **posición actual del puntero de lectura/escritura** dentro del archivo al que se refiere `nombre_logico`
`seek(nombre_logico, posición);`	permite **mover el puntero de lectura/escritura** a una `posición` específica dentro del archivo

Campo de un archivo

Un **campo** es la **unidad lógicamente más pequeña** y significativa de información dentro de un registro de un archivo.

- **Campo de longitud fija:** se define una cantidad fija de bytes para el campo. Si el dato no ocupa todos los bytes asignado al campo, se produce **fragmentación interna**, desperdiciando espacio de memoria. La ventaja es que conocemos donde comienza y donde termina físicamente cada campo dentro del archivo.
- **Campo de longitud variable:** el campo ocupa la cantidad de bytes que necesita el dato. No se conoce de antemano la dimensión ocupada. El campo se ajusta al tamaño del dato. Ventaja: No se produce fragmentación interna. Desventaja: No se sabe con precisión dónde comienza y dónde termina cada uno. **Genera fragmentación externa.**

Fragmentación

Fenómeno que ocurre cuando el espacio de almacenamiento se utiliza de manera ineficiente. Básicamente, se refiere a la **disposición no óptima del espacio**, lo que lleva a que una parte de la memoria o del disco quede inutilizable o sea difícil de asignar, incluso si hay suficiente espacio total disponible.
Existen dos tipos principales de fragmentación: **interna** y **externa**, ambas relacionadas con la utilización del espacio.

- **Fragmentación interna:** producida cuando a un registro se le asigna **más espacio del que realmente necesita o utiliza**. El espacio sobrante dentro de la unidad de asignación asignada queda sin usar, pero no puede ser utilizado por ningún otro elemento. Este tipo de fragmentación es frecuente con **registros de longitud fija**. Por ejemplo, un registro donde un campo `nombre` está definido para ocupar 50 caracteres. Si un nombre como "Juan Pérez" (que tiene 10 caracteres) se almacena en este campo, los 40 caracteres restantes dentro de ese campo específico son espacio reservado pero no utilizado, generando fragmentación interna.
- **Fragmentación externa:** espacio disponible entre dos registros, pero que es demasiado pequeño para poder ser reutlizado. Asociado a los **registros de longitud variable,** debido a que, cuando los registros tienen longitudes que cambian (porque se borran, se actualizan a un tamaño menor o se insertan nuevos registros de diferentes tamaños), se crean **huecos de espacio libre de distintas dimensiones** en el disco. Con el tiempo, estos huecos quedan dispersos y no contiguos.

Capítulo 3, algorítmica clásica sobre archivos

Se deben introducir los conceptos de `archivo maestro` y `archivo detalle`

- Archivo maestro: resume información sobre un dominio de problema específico.
- Archivo detalle: archivo que contiene novedades o movimientos sobre la información almacenada en el maestro.

Ambos archivos (maestro y detalle/s), deben estar ordenados por el mismo criterio.

Actualización de un archivo maestro con un archivo detalle

Las precondiciones a este problema son

- Existe un archivo maestro.
- Existe un único archivo detalle que modifica al maestro.
- Cada registro del detalle modifica a un archivo del registro maestro.
- Se descarta la posibilidad de generar altas en ese archivo.
- No todos los registros del maestro son necesariamente modificados.
- Cada elemento del maestro que se modifica es alterado *por uno y solo un* elemento del archivo detalle.
- Es precondición escencial que los archivos estén ordenados por igual criterio.
- El proceso de actualización finaliza cuando se termina de recorrer el archivo detalle.
- Es necesario buscar aquel registro del archivo maestro que se actualiza a partir del archivo detalle.

Proceso de generación de un nuevo archivo a partir de otros existentes, *merge*

Está vinculado con la generación de un archivo que resuma información de uno o varios archivos existentes. Recibe en nombre de *merge* y la principal diferencia es que el archivo maestro no existe y debe ser generado.

Corte de control

Se denomina corte de control al proceso mediante el cual la información de un arhivo es presentada en forma organizada de acuerdo con la estructura que tiene el archivo. Las precondiciones serán

- El archivo se encuentra ordenado por provincia, sucursal y vendedor.
- Se debe informar el total vendido en cada sucursal, ciudad, provincia así como el total final.
- En diferentes provincias pueden existir ciudades con el mismo nombre o en diferentes ciudades pueden existir sucursales con igual denominación.

Capítulo 4, eliminación de datos

Se denomina proceso de baja a aquel que permite quitar información de un archivo. Puede hacerse de 2 modos diferentes

- **Baja física:** consiste en borrar efectivamente la informacón del archivo, recuperando el espacio físico.
- **Baja lógica:** consiste en borrar la información del archivo, pero sin recuperar el espacio físico respectivo.

Baja física

Se realiza baja física sobre un archivo cuando un elemento es quitado del archivo y es reemplazado por otro elemento del mismo archivo, decrementando en uno su cantidad de elementos.

La ventaja de este método consiste en administrar un archivo de datos que ocupe el lugar mínimo necesario. La desventaja tiene que ver con la *performance* final de los algoritmos que implementan esta solución. Para la baja física, existen dos técnicas algorítmicas

- **Generar un nuevo archivo con los elementos válidos.**
    - Se recorrerá el archivo original, copiando al nuevo archivo toda la información, menos la correspondiente al elemento que se desea quitar. Una vez finalizado el proceso, se debe eliminar el archivo inciail del dispositivo de memoria secundaria. Un análisis de performance determina que este método necesita leer tantos datos como tenga el archivo original y escribirlos en el archivo nuevo. Suponiendo que el archivo tiene N registros, habrá N lecturas y N-1 escrituras.

      Finalizado el proceso, coexistirán los dos archivos entonces se deberá disponer en memoria secundaria el espacio suficiente para ambos.
- **Usar el mismo archivo, generando las reacomodamientos necesarios.**
    - En el caso de que se realice un borrado usando el mismo archivo, se requiere localizar el elemento, copiar sobre este registro el elemento siguiente y así sucesivamente.

Baja física vs. lógica en archivos directos

- **Baja lógica:** el registro no se elimina físicamente, sino que se marca de alguna manera (ej. con un flag, un valor especial) como "inactivo" o "borrado". El registro permanece en el archivo y sigue ocupando su espacio.
    - Es **menos eficiente en términos de espacio ocupado**, ya que el espacio no se libera inmediatamente y puede llevar a una acumulación de registros marcados como "borrados" que siguen consumiendo almacenamiento. Esto puede generar **fragmentación lógica**.
    - **Más eficiente en tiempo de ejecución** para la operación de baja individual, ya que solo implica una modificación simple del registro (actualizar un flag), sin necesidad de reorganizar el archivo.
- **Baja física:** implica la eliminación real del registro del archivo, liberando el espacio que ocupaba. A menudo requiere compactar el archivo para eliminar los "huecos" dejados por el registro borrado, lo que puede implicar mover otros registros.
    - Opción **más eficiente en términos de espacio ocupado**, ya que el espacio se recupera de forma inmediata y el archivo se mantiene compacto.

- Puede ser **menos eficiente en tiempo de ejecución** para la operación de baja, ya que reordenar el archivo para mantener la continuidad del espacio libre (o evitar fragmentación física) es una operación costosa, especialmente en archivos grandes.

Recuperación de espacio

El proceso de baja lógica marca como borrado un espacio de información de un archivo. A pesar de ser marcada, hay datos que siguen ocupando espacio. Habrá que recurrir a distintas técnicas para recuperar este espacio

- **Recuperación de espacio:** periódicamente utilizar el proceso de baja física para realizar un proceso de compactación del archivo.
- **Reasignación de espacio:** otra técnica alternativa posible consiste en recuperar el espacio, usando los lugares indicados como borrados para el alta de nuevos elementos del archivo.

  Consiste en reutilizar el espacio indicado como borrado para que nuevos registros se inserten en dichos lugares.

  Este proceso puede realizarse buscando los lugares libres desde el comienzo del archivo, pero se debe considerar que de esa manera sería muy lento. Por lo tanto la opción a usar será **lista encadenada invertida.** La **lista encadenada encadenada** es una estructura de datos fundamental utilizada para **mantener un registro de las posiciones que quedan disponibles** tras la eliminación de registros.

  Cuando un registro es eliminado de un archivo, su posición se marca como "libre". Estas posiciones libres no se dejan simplemente como huecos; en cambio, se "encadenan" entre sí. Esto significa que cada espacio libre contiene un puntero (una dirección o NRR) que apunta al siguiente espacio libre disponible en la secuencia.

  La **cabecera de la lista** es un puntero especial que guarda la dirección del **primer espacio libre disponible** para una futura asignación. A medida que se eliminan registros nuevos, sus posiciones se añaden a esta lista de libres. Comúnmente, la nueva posición liberada se añade al **principio de la lista** (comportamiento LIFO - Último en Entrar, Primero en Salir), de modo que la cabecera de la lista siempre apunta al espacio liberado más recientemente. Esto hace que la asignación de nuevos registros en estos espacios sea muy eficiente, ya que el sistema solo tiene que tomar el espacio apuntado por la cabecera.

  En el proceso de alta, primero se debe verificar la lista de espacios libres. En caso de encontrar alguno, el registro deberá insertarse en esa posición, actualizando la cabecera. Si el archivo no dispone de lugares para utilizar, el registro cabecera no tendrá ninguna dirección válida y en ese caso se agregará al final del archivo.

Eliminación con registros de longitud variable

Al recuperar espacio de registros borrados lógicamente para nuevas inserciones, la baja no basta con la eliminación superficial. Es crucial que el espacio disponible sea del **tamaño suficiente** para el nuevo elemento.

Para gestionar esto, se utiliza una **lista invertida** que, además de las posiciones, indica la cantidad de bytes disponibles en cada uno. La inserción de un nuevo elemento requiere **localizar el lugar más adecuado** dentro del archivo, para lo cual existen **tres estrategias principales** de selección de espacio.

- **Primer ajuste:** consiste en seleccionar el primer espacio disponible donde quepa el registro a insertar. Genera ***fragmentación interna.***
- **Mejor ajuste:** seleccionar el espacio más adecuado para el registro, el cual es aquel de menor tamaño donde quepa el registro a insertar. Genera ***fragmentación interna pero menor en comparación a primer ajuste.***
- **Peor ajuste:** consiste en elegir el espacio de mayor tamaño, asignando para el registro solo los bytes necesarios. Genera ***fragmentación externa.***

La técnica de primer ajuste será la más rápida caso contrario con el mejor y peor ajuste que son ineficientes, porque en ambos casos hay que recorrer toda la lista para encontrar el espacio más adecuado que una vez encontrado, mejor ajuste lo resolverá más rápido que peor ajuste.

Modificación con registros de longitud variable

Cuando se modifican registros de longitud variable, surge la posibilidad que la nueva información puede tener un tamaño menor, mayor o igual a la información actual. El problema es cuando la nueva es mayor que la actual y en este caso no será posible usar ese mismo espacio entones se reubicará.

Generalmente para evitar este análisis y facilitar la modificación, se divide el algoritmo en 2 etapas

1. Se da de baja al elemento de dato viejo.
2. Se inserta el nuevo registro con la política de recuperación de espacio determinada.

Capítulo 5, búsqueda de información: manejo de índices

Es fundamental **minimizar la cantidad de accesos a la memoria secundaria**. Cada acceso al disco es una operación relativamente lenta

Se presentan las siguientes estrategias básicas de búsqueda:

1. **Búsqueda Secuencial: i**mplica examinar cada elemento uno por uno hasta encontrar el deseado.
    - Es **muy poco eficiente**: su complejidad promedio es de **O(N/2)** comparaciones, lo que en términos de complejidad asintótica se simplifica a **O(N)** (lineal). Esto significa que, a medida que el número de elementos (N) crece, el tiempo de búsqueda aumenta proporcionalmente.
2. **Búsqueda Binaria:**
    - Es considerablemente **más eficiente**, con una complejidad de **O(log N)** (logarítmica). Esto implica que el número de operaciones de búsqueda crece de forma mucho más lenta a medida que aumenta N.
    - Sin embargo, su principal requisito y desventaja es que el **archivo debe estar siempre ordenado físicamente**. Mantener un archivo grande ordenado (especialmente tras inserciones y eliminaciones) puede ser un proceso muy costoso.

Surge la necesidad de una solución más flexible. La **idea central es crear una estructura auxiliar** que permita **acceder a la información como si estuviera ordenada, incluso si físicamente no lo está**. Esta estructura auxiliar se conoce como **Índice**.

Un *índice* es una herramienta porque encuentra registros en un archivo de una forma mas eficiente, consiste de un campo de llave/clave y un campo de referencia. Es una tabla que opera con un procedimiento que acepta información acerca de ciertos valores de atributos como entrada (llave) y provee como salida información que se encuentra en el archivo. Los índices nos permiten realizar búsqueda binaria.
El índice se puede implantar tanto sobre registros de longitud fija como registros de longitud variable. La estructura mas simple para implementar un índice es un árbol, especialmente árbol binario, porque nos permite realizar búsquedas binaria. De esta manera, con el índice podemos imponer orden en un archivo sin que realmente el archivo este ordenado.

- **Índices para claves candidatas**
    - **Clave candidata:** atributo o un conjunto de atributos que **identifica de forma única** cada registro, de manera similar a una clave primaria. Una tabla puede tener varias claves candidatas, pero solo una de ellas es finalmente seleccionada para ser la clave primaria por razones operativas o de diseño. Toda clave primaria fue una clave candidata, pero no toda clave candidata es necesariamente una clave primaria, ya que la clave primaria es la candidata *elegida* para ser el identificador principal.
- **Índices secundarios**
    - No siempre se realizan búsquedas de información utilizando la clave primaria. A menudo, los usuarios necesitan buscar registros basándose en otros atributos que pueden **admitir valores repetidos**. Por ejemplo,

al buscar una canción, es más común buscarla por su título, pero múltiples canciones pueden tener el mismo título.

Clave secundaria: atributo o un conjunto de atributos que **puede contener valores repetidos** en diferentes registros. A diferencia de una clave primaria o candidata, no garantiza la unicidad de los registros.

Un índice secundario es una **estructura de datos adicional** (generalmente un árbol, como un Árbol B+) que permite relacionar los valores de una **clave secundaria** con una o más claves primarias (o directamente con las ubicaciones físicas) de los registros donde aparece ese valor.

Su función principal es permitir **búsquedas eficientes por atributos no únicos o por atributos que no son la clave primaria**. Dado que una clave secundaria puede tener valores repetidos, un índice secundario para un valor dado de clave secundaria podría apuntar a múltiples registros.

Ejemplo: Un índice secundario para "título de canción" apuntaría desde "Bohemian Rhapsody" a todos los registros de canciones con ese título (por ejemplo, diferentes versiones o artistas).

Cuando empezamos a trabajar con índices, vamos a tener al menos dos archivos: un archivo de datos, y tantos archivos de índices como sean necesarios. Para que sea efectivo, debemos cargar el índice en memoria ram, ya que las operaciones sobre memoria RAM se miden en nanosegundos, mientras que las operaciones de los discos se mide en milisegundos. El índice se debe modificar luego de una alta o baja.

- **Creación de nuevos registros:** implicará agregar los datos al final del archivo de datos y su clave de forma ordenada en el archivo de índice.
- **Eliminación de un registro:** se borra del archivo de datos y saca del índice (en ambos archivos se pueden utilizar las técnicas para borrar información que ya vimos anteriormente.)
- **Actualización de registros:**
  - Si la modificación altera la clave: si se modifica la clave, seguro hay que modificar el archivo de datos, pero además se debe modificar el índice.
  - Si la modificación no altera la clave: en archivos con registros de longitud fija, si se modifica un registro, no hay cambio en el índice, por lo tanto, el registro se almacena en la misma posición física. Si el registro cambia de longitud se reubica en el archivo de datos y además se debe guardar la nueva posición inicial en el índice. En todo momento, el índice va a permitir búsqueda binaria, es de longitud fija, va a estar ordenado. Al permitir búsqueda binaria, encontrar la información se logra con una performance log2. Si puedo almacenar el índice en RAM, aun es más eficiente.

Capítulo 6, árboles: introducción

Sobre una B.D, casi el 80% de las operaciones consisten en búsqueda de datos. Es de gran importancia poder recuperar un dato requerido con la menor cantidad de accesos sobre disco.

Árboles binarios

Un árbol binario es una estructura de datos dinámica no lineal en la cual cada nodo puede tener a lo sumo 2 hijos y *tendrá sentido si está ordenado.*

La búsqueda se realiza a partir del nodo raíz y se recorre explorando hacia los nodos hoja, se chequea un nodo si es el deseado y la búsqueda finaliza si lo es y si no se continúa a izquierda o derecha descartando la mitad de los elementos restantes.

Tiene un $O(logN)$ donde N es la cantidad de elementos en el árbol.

La ventaja principal de los árboles binarios se da a la hora de la inserción de elementos ya que tiene una operatoria simple para insertarlo:

1. Agrega el nuevo elemento al final del archivo.

2. Busca al padre de dicho elemento.
3. Actualiza el padre haciendo referencia a la dirección del nuevo hijo.

Para quitar un elemento debe ser necesariamente un elemento terminal. Si no lo fuera, debe intercambiarse el elemento en cuestión con el menor de sus hijos mayores.

Se entiende por árbol balanceado a aquel árbol donde la *trayectoria de la raíz a cada uno de los hojas está representada por igual cantidad de nodos.* Es decir que todos los nodos hojas están a igual distancia del nodo raíz.

Árbol AVL

Los árboles balanceados en altura son árboles binarios cuya construcción de determina respetando que *la diferencia entre el camino más corto y el más largo entre un nodo terminal y la raíz no puede diferir en más de un determinado delta* y dicho delta es el nivel de balanceo en altura del árbol.

Cuando se haga una inserción habrá un rebalanceo lo cual tiene un gran costo computacional por lo que esta estructura se descarta porque no es viable ni eficiente.

Paginación de árboles binarios

La transferencia de datos entre la memoria secundaria y la memoria RAM no se realiza registro a registro individualmente. En su lugar, la información se transfiere en **bloques o conjuntos de datos más grandes**, conocidos como **páginas**.

Este concepto es de vital importancia cuando una estructura de datos grande, como un **árbol binario**, reside en un archivo en disco. Para optimizar su manejo:

- El árbol binario (o cualquier estructura similar) se divide lógicamente en estas "páginas". Cada página se corresponde con el tamaño de un bloque de transferencia de disco.
- Cuando un programa necesita acceder a una parte del árbol (por ejemplo, un nodo específico), el sistema no va al disco para traer solo unos pocos bytes o un solo registro. En su lugar, **transfiere una página completa** del disco a la RAM.
- Esta organización reduce drásticamente el **número total de accesos a disco**. Si los nodos del árbol están bien organizados en las páginas, es muy probable que una sola lectura de disco traiga a la memoria varios nodos que podrían ser necesarios para una búsqueda o recorrido posterior.

Al dividir un árbol binario en páginas, se logra una **búsqueda de datos significativamente más rápida** cuando estos están almacenados en memoria secundaria, ya que se minimizan las costosas operaciones de entrada/salida a disco.

Árboles multicamino

Un árbol multicamino es una estructura de datos en la cual cada nodo puede contener K elementos y K+1 hijos.

Órden de árbol multicamino: máxima cantidad de descendientes posibles de un nodo.

El orden de un árbol binario es 2, de uno ternario será 3 y un multicamino será orden M, o sea un máximo de M descendientes.

Un multicamino representa otra forma de resolver el concepto de paginación.

Capítulo 7, familia de árboles balanceados

Árboles B (balanceados)

Los árboles B son árboles multicamino con una construcción especial que permite mantenerlos a bajo costo, los cuales tienen ciertas propiedades básicas

- Cada nodo del árbol puede contener como *máximo M descendientes y M-1 elementos.* Si el orden de un árbol B es 256, indica que la máxima cantidad de descendientes de cada nodo es 256.
- La raíz no posee descendientes o posee al menos 2.
- Un nodo con X descendientes directos, contiene X-1 elementos. Determina que cualquier nodo que posee una cantidad determinada de hijos por ejemplo 256, necesariamente debe contener 255 elementos.
- Los nodos terminales tienen como mínimo [M/2] - 1 elementos y como máximo M-1 elementos. Al ser M el orden del árbol, la máxima cantidad admisible de elementos será M-1 pero además, la mínima cantidad de elementos es uno menos que la parte entera de la mitad del orden. Así si 256 es el orden un nodo terminal deberá posee 127 elementos como mínimo y 255 como máximo.
- Los nodos que no son terminales ni raíz tienen, como mínimo [M/2] elementos.
- Todos los nodos terminales se encuentran al mismo nivel. Por lo tanto esto establece el balanceo del árbol.

Operaciones sobre árboles B

- **Creación:** un árbol balanceado tiene como principal característica que todos los nodos terminales se encuentran a la misma distancia del nodo raíz. El proceso inicia con una estructura vacía y se comenzará a agregar los elementos. Si el nodo raíz tiene lugar, se procederá a insertarlos en ese nodo teniendo en cuenta que los elementos de datos deben quedar ordenados en ese nodo.

  La llegada de un nuevo elemento provocará un *overflow* en el nodo. Esto significa que en el nodo no hay capacidad disponible para almacenar un nuevo elemento de datos. Ante la ocurrencia de overflow, el proceso será el siguiente

  1. Se crea un nodo nuevo.
  2. La primera mitad se mantiene en el nodo viejo.
  3. La segunda mitad se traslada al nuevo nodo.
  4. La menor de las claves de la segunda mitad se promociona al nodo padre.
- **Búsqueda en un árbol B:** el proceso de búsqueda coincide con la primera parte del proceso mostrado. El primer paso de la inserción consiste en localizar el nodo que debería contener al nuevo elemento. Comenzando desde la raíz se procede a buscar el elemento en cuestión. En caso de encontrarlo en dicho nodo, se retorna una condición de éxito. Caso contrario, se procede a buscar un nodo inmediato siguiente.
- **Eliminación en árboles B:** existen 2 situaciones en el proceso de baja
  1. Cuando al borrar el elemento del nodo terminal, la cantidad de elementos restantes no está por debajo de la cantidad mínima [M/2] - 1. En este caso no se genera *underflow* y el proceso termina.
  2. Al eliminar un elemento, se produce *underflow*. En este caso, el nodo al que se le quita un elemetno deja de cumplir la condición de tener al menos [M/2] - 1 elementos y se presentarán 2 situaciones
     - Redistribución: apliado si un nodo hermano adyacente tiene suficientes elementos como para “prestar” uno, es decir, tiene más de [M/2] - 1 elementos. El proceso a seguir será el siguiente
       1. Se “toma prestado” un elemento del hermano más cercano.
       2. El elemento que separa al nodo afectado y a su hermano en el nodo padre se “baja” al nodo que sufrío el underflow, es una especie de corrimiento.
       3. El elemento prestado del hermano se “promociona” al nodo padre, reemplazando al elemento que se bajó.
     - Concatenación/ fusión: aplicada si el nodo hermano adyacente no tiene elementos para prestar, su tiene [M/2] - 1 elementos. Los pasos serán
       1. El nodo que sufrió el underflow se fusiona con su nodo hermano adyacente.

2. El elemento del nodo padre que separaba a estos dos nodos también se "baja" y se incorpora al nuevo nodo fusionado.

Como resultado se forma un único nodo más grande que contiene todos los elementos de los dos nodos originales más el elemento “bajado” del padre.

Árboles B*

Representan una variante sobre los árboles B. Es una variante para la distribución a la hora de overflow.

En B* si la cantidad de niveles del árbol crece lentamente, habrá mejor performance.

Si se aplica el concepto de redistribución, cuando un nodo se completa, reubica sus elemetnos usando un nodo adyacente hermano. Cuando no sea posible esta redistribución, se estará ante una situación donde tanto el nodo hermano adyacente como en el que se genera overflow están completos. Abre la posibilidad de dividir los 2 nodos completos y generando 3 nodos completos en 2/3 partes.

Un árbol B* es un árbol balanceado con las siguientes propiedades:

- Cada nodo del árbol puede contener como máximo M descendientes y M-1 elementos.
- La raíz no posee descendientes o tiene al menos 2.
- Un nodo con X descendientes, tiene X-1 elementos.
- Los nodos terminales tienen como mínimo [2M -1/ 3] - 1 elementos y como máximo M-1 elementos.
- Los nodos que no son terminales ni raíz tienen como mínimo [2M - 1]/ 3 descendientes.

Se puede observar que cuando un árbol B* tiene un solo nivel y este se completa, no puede redistribuirse dado que la raíz no tiene hermanos. Entonces la propiedad que indica la mínima cantidad de elementos tiene una excepción de tratamiento, al dividir el nodo raíz y generar 2 hijos. En este caso, los 2 nodos hijos generados solamente completarán su espacio a la mitad. Esta situación se considera como la única excepción admitida para el tratamiento de las propiedades del árbol B*.

La operación de búsqueda sobre un árbol B* es similar a la presentada anteriormente para árboles B.

En conclusión, su característica distintiva es que los nodos (excepto la raíz) deben estar llenos en un **factor de ocupación mínimo más alto** (generalmente 2/3 o más) antes de que se produzca una división. Cuando un nodo se llena, primero intenta redistribuir sus claves con un nodo hermano adyacente antes de dividirse, lo que reduce la frecuencia de las divisiones y las operaciones de E/S.

Árbol B+

Es un árbol multicamino que

- Cada nodo del árbol puede contener como máximo M descendientes y M-1 elementos.
- La raíz no posee descendientes o posee al menos 2.
- Un nodo con X descendientes contiene X-1 elementos.
- Los nodos terminales tienen como mímo [M-2] - 1 elemetnos y como máximo M-1 elementos.
- Los nodos que no son terminales ni raíz tienen como mímo M-2 descendientes.
- Todos los nodos terminales se encuentran al mismo nivel.
- Los nodos terminales representan un conjunto de datos y son enlazados entre ellos.

Hay similitudes con un árbol B excepto por la última propiedad, la cual establece que para poder realizar acceso secuencial ordenado a todos los registros del archivo es necesario que cada elemento esté almacenado en un nodo terminal. Así, los árboles B+ diferencian los elementos que constituyen datos de aquellos que son separadores.

En el momento en el que el nodo se satura, se produce una división. Se genera un nuevo nodo terminal donde se redistribuyen los elementos. Se debe notar que la clave utilizada como separador es la misma que está contenido en el nodo terminal, es decir, se usa una copia del elemento y no el elemento en sí.

El **proceso de creación** sigue los lineamientos de los árboles B y en caso de dividir un nodo no terminal, se debe promocionar hacia el padre el elemento en sí y no una copia del mismo.

Entonces, para borrar un elemento de un árbol B+, siempre se borra de un nodo terminal y si hubiese una copia de ese elemento en un nodo no terminal.

Conclusiones

Característica	Árbol B	Árbol B*	Árbol B+
Definición	Estructura balanceada con múltiples claves y punteros por nodo.	Optimización del árbol B para mejor rendimiento y espacio.	Variante del Árbol B optimizada para búsquedas secuenciales.
Ubicación de los datos	Pueden residir tanto en nodos internos como en los terminales.	Ídem árbol B	**Exclusivamente en los nodos hoja**. Los nodos internos solo contienen claves de índice.
Optimización principal	Búsquedas eficientes en sistemas de almacenamiento en disco, mantener datos ordenados.	Mejora el rendimiento general y la **utilización del espacio** (menos divisiones).	**Búsquedas secuenciales** (hojas enlazadas). Eficiencia en sistemas de archivos y bases de datos.
Reglas de llenado/ división	Divide el nodo cuando se llena (M-1 elementos), promocionando la clave media.	Los nodos deben estar llenos a un factor mínimo más alto (generalmente **2/3 o más**) antes de dividirse. Intenta **redistribuir con un hermano** antes de dividir.	Similar al Árbol B en la gestión de nodos internos; las hojas se enlazan y manejan de forma particular.
Punteros en hojas	Los nodos hoja contienen claves y, a veces, punteros a los datos.	Los nodos hoja contienen claves y, a veces, punteros a los datos.	Todos los **nodos hoja están enlazados secuencialmente** entre sí.

¿Por qué un árbol solo puede ser de un tipo específico?

Un árbol solo puede ser de un tipo (B, B+, o B*) y no una combinación simultánea de ellos porque sus características estructurales y reglas de operación son **mutuamente excluyentes**:

- **Árbol B vs. B+:** la principal diferencia reside en la **ubicación de los datos**. Un árbol B permite datos tanto en nodos internos como en hojas, mientras que un árbol B+ los restringe exclusivamente a las hojas, manteniendo los nodos internos solo como guías de índice. Estas definiciones son incompatibles en una misma estructura.
- **Árbol B vs. Árbol B*:** aunque el árbol B* es una mejora y una variante del árbol B, implementa **reglas distintas y más complejas para el llenado y la división** de los nodos (ej., un factor de llenado mínimo más alto y el intento de redistribución con un hermano antes de cualquier división). No son simplemente fases, sino algoritmos de mantenimiento y estructuras intrínsecas con lógicas diferentes que no pueden coexistir simultáneamente en la misma implementación.

Hashing

Capítulo 8, Dispersión, hashing

Un archivo directo es un archivo en el cual cualquier registro puede ser obtenido sin acceder a otros registros anteriormente. En un archivo serie, un registro está disponible solo cuando el registro predecesor fue procesado.

Conceptos de dispersión- métodos de búsqueda más eficientes

Hashing es un método de organización y acceso a datos que busca **optimizar drásticamente la eficiencia de recuperación de información**, apuntando a lograr, en promedio, un solo acceso para localizar un registro. Supera en velocidad de acceso promedio a estructuras como los árboles balanceados.

El corazón del hashing es una **función hash**. Esta función toma la clave de un registro como entrada y transformará esa clave en un valor hash, que es un número entero. El valor hash generado es la base para determinar la **dirección física o lógica** exacta donde el registro debe ser almacenado. En esencia, la función hash es directamente responsable de obtener esta dirección de almacenamiento del registro.

A pesar de sus ventajas, cuenta con ciertas limitaciones las cuales son

- *No es posible aplicar hashing en registros de longitud variable.*
- No es posible obtener un orden lógico.
- No es posible tratar con claves duplicadas.

Tipos de dispersión

Hashing presenta 2 alternativas para su implentación: tratamiento de espacio en forma estática o forma dinámica.

- **Hashing con espacio de direccionamiento estático:** política donde el espacio disponible para dispersar los registros de un archivo está fijado previamente. Así, la función de hash aplicada a una clave da como resultado una dirección física posible dentro del espacio disponible para el archivo.
- **Hashing con espacio de direccionamiento dinámico:** política donde el espacio disponible para dispersar los registros de un archivo aumenta o disminuye en función de las necesidades que en cada momento tiene el archivo.

Parámetros de hashing

- **Función de hash:** esta función puede verse como una caja negra que recibe como entrada una clave y produce una dirección de memoria.

  La función de hash debe retornar como resultado una dirección que representa una llave primaria de un registro, en otro valor dentro de un determinado rango. Si usando la función de hash, se obtiene la misma dirección significa que se debe almacenar en el mismo lugar y se generará una **colisión** entre ambos registros, en este caso serán denominados *sinónimos.*

  Al utilizar el método de hashing, la función hash convierte una clave en una dirección de almacenamiento. Sin embargo, es posible que **dos o más claves diferentes generen la misma dirección de hash**. Esta situación se denomina **colisión**.

  Cuando se produce una colisión, las claves que se mapean a la misma dirección se intentan almacenar en el mismo "lugar" o "espacio" asignado a esa dirección. Este lugar es a menudo un **nodo**, que tiene una **capacidad limitada** para albergar múltiples registros.

  - **Almacenamiento en el nodo:** Si el nodo tiene espacio disponible, las llaves colisionadas pueden ser almacenadas allí.
  - **Overflow:** no obstante, la capacidad de cada nodo es finita. Si, a pesar de la capacidad del nodo, un nuevo registro colisionado ya no puede ser almacenado en esa dirección porque el nodo está completamente lleno, se dice que el nodo está en **overflow**. Ante un overflow, se activan mecanismos de resolución de colisiones adicionales.

  Así, es necesario encontrar una solución a este problema así, se presentan 2 alternativas

  - Elegir un algoritmo de dispersión perfecto que no genere colisiones. Debe asegurar que dadas 2 claves diferentes, siempre se obtendrán 2 direcciones físicas pero esto resulta difícil.

- Minimizar el número de colisiones a una cantidad aceptable y de esta manera tratar dichas colisiones como una condición excepcional. Existen diferentes modos de reducir el número de colisiones, las cuales serán
  - Distribuir los registros de la forma más aleatoria posible, las colisiones se presentan cuando 2 o más claves compiten por la misma dirección. Para ello, se debe buscar una función de dispersión que distribuya su resultado de la forma más aleatoria posible.
  - Utilizar más espacio en disco, si se intentan distribuír los registros 10 registros en 10 lugares, significa que hay un lugar disponible para cada uno. Con lo cual las posibilidades de colisión son altas. Ahora bien, si se dispone de 100 lugares para 10 registros, se tendría un archivo con 10 direcciones posibles para cada registro.

    Esto disminuirá el número de colisiones pero la desventaja es el desperdicio de espacio.

- **Tamaño de cada nodo de almacenamiento:** dentro de los parámetros que afectan la eficiencia del método de hashing estrá presente el tamaño o capacidad de cada nodo. Para determinar el tamaño ideal de un nodo, se realiza el mismo análisis que en árboles. Es así que la capacidad del nodo queda terminada por la posibilidad de transferencia de información en cada operación de entrada/ salida desde RAM hacia disco y viceversa.
- **Densidad de empaquetamiento:** se define la *D.E como la relación entre el espacio disponible para el archivo de datos y la cantidad de registros que integran dicho archivo.*

  Si se debería esparcir 30 registros entre 10 direcciones con capacidad de 5 registros por cada direcci{on, la D.E sería 0,6 o 60% (30/5) * 10. Cuando mayor sea la D.E, mayor sea la posibilidad de colisiones, dado que en ese caso se dispone de menos espacio para esparcir registros.

  Por el contrario, si la D.E se mantiene baja, se despercidia espacio en el disco, dado que se usa menor espacio que el reservado, generando fragmentación.

  La D.E no es constante, al inicio la D.E será baja y puede continuar creciendo hasta una D.E de 99%.
- **Métodos de tratamiento de desbordes (*overflow*):** un overflow ocurre cuando un registro es direccionado a un nodo que no dispone de capacidad para almacenarlo. Deben realizarse 2 acciones: encontrar lugar para el registro en otra dirección y asegurarse de que el registro posteriormente sea encontrado en esa nueva dirección.

  Aunque la función de hash sea eficiente y aún con D.E baja, es probable que las colisiones produzcan overflow o aún con la D.E relativamente baja, es probable que las colisiones produzcan overflow o saturación.

  Se debe contar con algún método para reubicar aquellos registros.

Se presentan 4 métodos:

1. Saturación progresiva.
2. Saturación encadenada.
3. Doble dispersión.
4. Área de desbordes por separado.

Saturación progresiva

El método consiste en almacenar el registro en la dirección siguiente más próxima al nodo donde se produce la saturación.

Saturación progresiva encadenada

La saturación progresiva encadenada presenta otra variante al tratamiento de la saturación. En líneas generales, el método funciona igual a su predecesor. Un elemento que se intenta ubicar en una dirección completa es direccionado a la inmediata siguiente con espacio disponible.

La diferencia radica en que, una vez localizada la nueva dirección, esta se encadena con la dirección base inicial, generando una cadena de búsqueda de elementos.

Doble dispersión

El método consiste en disponer de 2 funciones hash. La primera obtiene la dirección donde el registro será ubicado. De producirse un overflow, se utilizará la segunda función de hash la cual no retorna una dirección, si no que su resultado es un desplazamiento. Este esplazamiento se suma a la dirección base obtenida en la primera función, generando así la nueva dirección donde se intentará ubicar el registro.

La doble dispersión tiende a esparcir los registros en saturación a lo largo del archivo de datos.

Área de desborde por separado

Ante la ocurrencia de overflow, los registros son dispersados en nodos que no se corresponden con su dirección base original. Así, a medida que se completa un archivo por dispersión puede existir muchos registros ocupando direcciones que originalmente no les corresponden, disminuyendo la performance inicial.

- **Ventaja:** mantiene las cubetas principales más limpias y reduce la formación de "*clusters*" de datos contiguos en la tabla principal, que es un problema en la saturación progresiva.
- **Desventaja:** Implica un **doble acceso** a disco si el registro está en el área de desborde (primero a la tabla principal, luego al área de desborde). Además, a medida que el área de desborde se llena, la búsqueda en ella puede volverse lineal, lo que disminuye la performance inicial del hashing.

Glosario

> Memoria secundaria: tipo de **almacenamiento no volátil** (los datos persisten al apagar el dispositivo) de gran capacidad y menor velocidad que la memoria principal.

> Independencia física: capacidad de **modificar la organización o estructura física** de los datos almacenados **sin que esto afecte la vista lógica o los programas (algoritmos)** que acceden a esos datos.

> Clusters: unidad de almacenamiento más pequeña que un sistema operativo puede asignar a un archivo en un disco duro o cualquier otro medio de almacenamiento.